NAME :ANA MAHARANI PUTRI

STUDENT ID : 26010340

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

Breadth-First Search (BFS) treats the graph as unweighted for path selection. It expands states in FIFO order, so it finds a path with the fewest edges (hops), not necessarily the lowest total distance. Edge distances therefore do not affect its ordering. If the objective is to minimize total distance, Uniform-Cost Search (UCS) is the appropriate uninformed algorithm.

2. Which two lines change when BFS is turned into DFS, and which of them makes DFS order dependent?

The frontier behavior changes from a FIFO queue to a LIFO stack: the frontier is initialized with a stack-like structure, and the next state is removed from the end (for example, pop-left for a queue becomes pop for a list/stack). The LIFO removal is the key algorithmic change. DFS's exact traversal order also depends on the order in which neighbors are iterated and pushed onto the stack; reversing that order can produce a different traversal. The precise line numbers depend on the implementation used in the lab.

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

A state may be generated through an expensive path before a cheaper path to the same state is discovered. If UCS stops as soon as the goal is generated, it can return a non-optimal route. Testing the goal when it is popped from the priority queue ensures that, with non-negative edge costs and the standard UCS procedure, the goal is selected at its lowest path cost.

4. Lab 1 reports expanded and runtime_ms. Which should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

Use the number of expanded states as the primary comparison. It measures search effort and is generally more comparable across computers. Runtime in milliseconds depends on hardware, system load, interpreter overhead, and other implementation details, so it can vary even when the search work is similar. Runtime can still be reported as a secondary measure.

5. Which quantity does GBFS ignore that A* uses, and which failure in Lab 2 does that explain?

Greedy Best-First Search (GBFS) prioritizes the heuristic h(n) and ignores the path cost already incurred, g(n). A* prioritizes f(n) = g(n) + h(n), accounting for both the cost so far and the estimated remaining cost. This explains why GBFS can return a longer, more expensive route even when its search effort is small. In the supplied results, GBFS is suboptimal for Q1 (12,500.50 km versus 11,461.35 km; gap 9.07%) and Q3 (15,811.29 km versus 14,813.32 km; gap 6.74%).

6. Frontier states X and Y have (g, h) = (8, 2) and (3, 6). Which does UCS expand first? GBFS? A*?

UCS chooses the lowest g: Y, because 3 < 8. GBFS chooses the lowest h: X, because 2 < 6. A* chooses the lowest f = g + h. X has f = 8 + 2 = 10, while Y has f = 3 + 6 = 9, so A* chooses Y.

7. Why must h be expressed in the same units as the edge costs?

A* adds g(n), the accumulated path cost, to h(n), the estimated remaining cost. The quantities must use compatible units for the sum to be meaningful and for the heuristic to be compared fairly with path costs. For example, if edge costs are in kilometres, h should also estimate kilometres. Mixing kilometres with minutes or an arbitrary unconverted scale can distort priorities and invalidate the intended optimality guarantees.

8. Lab 2 computes h* by running Dijkstra on the reversed graph from the goal. Why reversed?

The desired h*(n) is the true shortest-path cost from every node n to the goal. Reversing every directed edge turns each original path from n to the goal into a path from the goal to n in the reversed graph, with the same total cost. Running Dijkstra once from the goal on that reversed graph therefore computes the shortest cost-to-go for all nodes, assuming non-negative edge weights.

9. Weighted A* with w = 1.5 changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

Weighted A* typically prioritizes $f_w(n) = g(n) + w \cdot h(n)$. Increasing the heuristic weight can change which frontier state is preferred, so the KUL→KEF search selected a different, more expensive route. For SIN→EZE, the weighted search retained the same route as UCS and ordinary A*—SIN → JNB → GRU → EZE—with a cost of 17,817.36 km and a 0% gap. This indicates that, for this query, increasing the heuristic weight did not change the winning route decisions; the optimal route remained preferred despite the stronger heuristic influence. The result alone does not prove that the route is uniquely optimal or that every alternative was unaffected.

Relevant results from the supplied tables

GBFS versus optimal cost: Q1: 12,500.50 km versus 11,461.35 km (9.07% gap); Q2: 17,817.36 km versus 17,817.36 km (0% gap); Q3: 15,811.29 km versus 14,813.32 km (6.74% gap).

Weighted A* on SIN→EZE: 17,817.36 km, same route as UCS and A*, with a 0% gap.